

auction.c

```
#include <stdio.h>
#include "types.h"
#include "functions.h"

/* Rule-LK 19: opening bid is 50% of market value. A market decline
lowers the starting price by 25% (Rule-LK 32/34) - group-scoped.
Railways and utilities use currentMarketValue too, so their value
modifiers (Port Expansion, Water Shortage, ...) apply the same way
they do for colour-group properties. */
int getAskingValue(GameState game[], int propIndex)
{
    int value;
    int group;

    value = currentMarketValue(game, propIndex);

    group = groupOf(game, propIndex);

    value = (value * modifierMultiplier(game, MOD_AUCTION_PRICE, group, -1)) / 100;

    return value;
}

/* Rule-LK 19-23: runs an auction among all solvent players. A
player who declines to bid is out permanently, so the pool of
bidders can only shrink - guaranteeing the loop finishes.
sellerIndex >= 0 (Anti-Speculation Act forced sale) sends the
proceeds to that player; sellerIndex == -1 pays the Bank. */
void runAuction(GameState game[], int propIndex, int sellerIndex)
{
    int active[MAX_PLAYERS];
    int activeCount;
    int i;
    int currentBid;
    int highBidder;
    int candidateBid;
    int safetyRounds;
    char moneyBuf[32];

    printf("\n*** AUCTION ***\n");
    printf("Property : %s\n", game[0].board[propIndex].name);

    currentBid = getAskingValue(game, propIndex) / AUCTION_OPENING_DIVISOR;

    formatLKR(currentBid, moneyBuf);
    printf("Opening Bid : LKR %s\n", moneyBuf);

    highBidder = -1;
    activeCount = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        active[i] = !game[0].players[i].bankrupt;

        /* Anti-Speculation Act (Rule-LK 8): the forced seller must
        actually part with the property, so they never bid. If the
        seller were allowed to bid, they could win the auction back
        and merely pay the bid to themselves (net-zero cost), which
        defeats the forced sale. */
        if(i == sellerIndex)
```

```

        active[i] = 0;

    if(active[i])
        activeCount++;
}

safetyRounds = 0;

while(activeCount > 1 && safetyRounds < AUCTION_MAX_ROUNDS)
{
    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(!active[i])
            continue;

        if(activeCount <= 1)
            break;

        candidateBid = currentBid + AUCTION_BID_INCREMENT;

        if(willingToBid(game, i, propIndex, candidateBid))
        {
            currentBid = candidateBid;
            highBidder = i;

            formatLKR(currentBid, moneyBuf);
            printf("%s bids LKR %s.\n", game[0].players[i].name, moneyBuf);
        }
        else
        {
            active[i] = 0;
            activeCount--;

            printf("%s withdraws.\n", game[0].players[i].name);
        }
    }

    safetyRounds++;
}

/* If exactly one bidder is still active, give them one final
   chance to bid at the next increment before awarding. */
if(activeCount == 1 && highBidder == -1)
{
    int remaining;

    remaining = -1;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(active[i])
        {
            remaining = i;
            break;
        }
    }

    if(remaining != -1)
    {
        candidateBid = currentBid + AUCTION_BID_INCREMENT;

        if(willingToBid(game, remaining, propIndex, candidateBid))
        {

```

```

        currentBid = candidateBid;
        highBidder = remaining;

        formatLKR(currentBid, moneyBuf);
        printf("%s bids LKR %s.\n",
                game[0].players[remaining].name, moneyBuf);
    }
}

if(highBidder == -1)
{
    int lastActive;

    /* One player never got a chance to bid (e.g. the first active
       player declined while only two were in) - they may take the
       property at the opening bid. */
    lastActive = -1;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(active[i])
        {
            lastActive = i;
            break;
        }
    }

    /* No one left to bid, or they decline the opening price */
    if(lastActive == -1 || !willingToBid(game, lastActive, propIndex, currentBid))
    {
        printf("No bids received. %s remains with the Bank.\n",
                game[0].board[propIndex].name);
        return;
    }

    highBidder = lastActive;
}

/* Always format the final winning price so the display is correct
   regardless of which code path set highBidder. */
formatLKR(currentBid, moneyBuf);

payMoney(game, highBidder, currentBid);

if(sellerIndex >= 0)
    receiveMoney(game, sellerIndex, currentBid);

game[0].board[propIndex].property.owner = highBidder;

adjustOwnedCount(game, highBidder, propIndex, 1);

printf("%s wins the auction for LKR %s.\n",
        game[0].players[highBidder].name, moneyBuf);
}

```